Selenium WebDriver

Theme: User Login Automation System

Sections A, B and C

Section A : Conceptual Questions

Q1

Why is API testing preferred when there is no UI?

When the UI is not ready we test the API directly. This way we can check if the logic is working properly without waiting for the frontend to be built. It is faster and we can find bugs early. It is also easy to automate.

Q2

During automation execution login validation fails intermittently. Explain how control statements can be used to handle different outcomes.

We can use if and else to handle different results. If login is successful we print a pass message. If it fails we go to the else part and print a fail message. This way the script knows what to do in each case.

Q3

Error messages on the login page change dynamically. Explain how string handling helps validate correct messages.

Sometimes the error message is a little different each time. So instead of checking the full message using equals we use contains to check only the important part. This works even if the message changes slightly.

Q4

Explain how object-oriented concepts improve maintainability in automation frameworks.

We create a separate class for each page. All elements and methods for that page are kept inside that class. If something changes on the page we only update that one class. This keeps the code clean and easy to manage.

Q5

An automation script fails due to invalid input. Explain how exception handling helps prevent script termination.

If we use try and catch the script will not stop when an error happens. The error is caught in the catch block and we print a message. The remaining tests can still run normally.

Section B : Practical Tasks

1. Conditional Login Validation

I wrote a program that takes username and password from the user and checks if they are correct.

```
import java.util.Scanner;

public class LoginCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter username: ");
        String user = sc.nextLine();
        System.out.print("Enter password: ");
        String pass = sc.nextLine();
        if (user.equals("admin") && pass.equals("Admin@123")) {
            System.out.println("Login Successful");
        } else {
            System.out.println("Access Denied");
        }
    }
}
```

2. Multi-User Testing

I stored multiple usernames and passwords in a HashMap and used a loop to test each one.

```
import java.util.HashMap;
import java.util.Map;

public class MultiUserTest {
    public static void main(String[] args) {
        HashMap<String, String> users = new HashMap<>();
        users.put("admin", "Admin@123");
        users.put("testuser", "wrongpass");
        users.put("john", "pass123");
        for (Map.Entry<String, String> entry : users.entrySet()) {
            if (entry.getKey().equals("admin") &&
entry.getValue().equals("Admin@123")) {
                System.out.println(entry.getKey() + " : Login Successful");
            } else {
                System.out.println(entry.getKey() + " : Access Denied");
            }
        }
    }
}
```

3. Method Overloading

I created three login methods with the same name but different inputs to show method overloading.

```
public class LoginOverload {
    public static void login(String username, String password) {
        System.out.println("Login with username and password");
    }
    public static void login(String username, String password, int otp) {
```

```
        System.out.println("Login with OTP: " + otp);
    }
    public static void login(long phone, String password) {
        System.out.println("Login with phone: " + phone);
    }
    public static void main(String[] args) {
        login("admin", "Admin@123");
        login("admin", "Admin@123", 1234);
        login(9876543210L, "Admin@123");
    }
}
```

4. Exception Handling

I made a custom exception and used try-catch so the program does not stop when login fails.

```
class InvalidLoginException extends Exception {
    public InvalidLoginException(String msg) {
        super(msg);
    }
}

public class ExceptionDemo {
    static void login(String u, String p) throws InvalidLoginException {
        if (!u.equals("admin") || !p.equals("Admin@123")) {
            throw new InvalidLoginException("Wrong credentials");
        }
        System.out.println("Login Successful");
    }
    public static void main(String[] args) {
        try {
            login("admin", "Admin@123");
            login("user", "wrong");
        } catch (InvalidLoginException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

5. File I/O Logging

This program saves the login result with time into a file called results.txt.

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.time.LocalDateTime;

public class LoginLogger {
    public static void main(String[] args) throws Exception {
        BufferedWriter bw = new BufferedWriter(new FileWriter("results.txt"));
        String time = LocalDateTime.now().toString();
        bw.write("[" + time + "] admin : Pass");
        bw.newLine();
```

```
            bw.write("[" + time + "] testuser : Fail");
            bw.newLine();
            bw.close();
            System.out.println("Results saved to results.txt");
        }
    }
```

Section C : Mini Capstone Project

1. Test Planning Document

Category	Details
Scope	Testing login feature using Java. Covers correct login, wrong login, and logout.
Tools Used	Java, Selenium WebDriver, TestNG, ChromeDriver.
Test Data	Username and password stored in HashMap. Both correct and wrong values are tested.
Authentication	Valid credentials are checked. Wrong credentials throw an exception.
Risks	Error message on page may change. Browser version may not match ChromeDriver.
Execution	Tests run one by one. If any test fails the error is caught and saved to results.txt.

2. Requirements Traceability Matrix

Req ID	Requirement	Test / Method	Type	Status
REQ-01	User can log in with correct credentials	login(String, String)	Positive	Covered
REQ-02	Wrong credentials are blocked	InvalidLoginException	Negative	Covered
REQ-03	OTP login should work	login(String, String, int)	Positive	Covered
REQ-04	Phone number login should work	login(long, String)	Positive	Covered
REQ-05	Login results saved to file	LoginLogger	Logging	Covered

3. End-to-End Test Scenario

Application Launch -> Login Validation -> Error Handling -> Logout

Step	Action	Method	Expected Result
1	Open browser and go to login page	ChromeDriver launch	Login page is shown.

2	Enter correct username and password	login(String, String)	Login is successful.
3	Enter wrong password	catch InvalidLoginException	Error is caught. Program does not stop.
4	Save result to file	BufferedWriter	results.txt is created with pass and fail.
5	Click logout	driver.findElement click	Login page is shown again.

4. Test Summary Report

Section	Details
Total Tests Run	5
Pass / Fail	All 5 Passed
Exceptions	Wrong login exception was caught properly. Program continued.
File Log	results.txt was created with time and pass or fail for each test.
Defects	No defects found.
Recommendation	APPROVED. All tests passed. Ready for next step.